End-to-End Automation Framework for BlazeDemo Flight Booking Application

Problem Statement:

BlazeDemo is a publicly available demo website designed to mimic a real-world flight booking platform. Your task is to **design and implement a modular, scalable automation testing framework** using **Java, Selenium WebDriver, and TestNG** to validate all key functionalities.

This project simulates how automation frameworks are structured in companies and provides hands-on experience building them from scratch.

Objective:

- Understand the structure of a flight booking application.
- Create a reusable Selenium framework using Java and TestNG.
- Automate real user journeys.
- Handle positive and negative test cases.
- Use Maven to manage dependencies.
- Prepare for future integrations like CI/CD and advanced reporting.

Application Under Test:

URL: https://blazedemo.com

Key Pages

Page	Description
Home Page	Choose departure and destination cities.
Reserve Page	Lists available flights.
Purchase Page	Enter passenger and payment information.
Confirmation Page	Displays booking confirmation.

Step-by-Step Implementation Guide:

Step 1: Analyze the Application

Understand workflows: select cities → search → choose flight → purchase → confirm booking.

Step 2: Set Up Project

- Create a **Maven project** in your IDE (e.g., IntelliJ IDEA, Eclipse).
- Add dependencies to pom.xml:

Step 3: Design Framework (Page Object Model)

Packages

com.blazedemo.pages
com.blazedemo.tests
com.blazedemo.utils (optional)

Page Classes

Class	Actions Covered
HomePage.java	Select cities, click "Find Flights".
ReservePage.java	Choose a flight. PurchasePage.java
	Enter passenger C payment info.
ConfirmationPage.java	Validate success message.

Step 4: Write a Basic Smoke Test

- Create FlightBookingTest.javaunder testspackage.
- Automate booking a flight from Boston to New York.
- Assert that the confirmation page is displayed.

Step 5: Add Data-Driven Testing

- Use @DataProviderin TestNG to provide multiple user data sets (e.g., different names, addresses, card details).
- Verify each booking is successful.

Step 6: Add Negative Test Cases

Scenario	Expected Outcome
Submit blank credit card	Confirmation page should not appear.
Submit non-numeric credit card	Proper validation or error behavior.
Leave required fields blank	Proper validation or error behavior.
Same departure C destination city	Booking should not proceed.

Step 7: Organize Tests in Suite

- Create testng.xml.
- Add all test classes and define groups (e.g., smoke, functional, negative).

Step 8: Run C Debug

- Run via IDE or CLI (mvn test).
- Fix waits, locators, or timeouts as needed.
- Add assertions for validations.

Step 9: Optimize C Maintain

- Move driver setup to a Base class.
- Use @BeforeMethod, @AfterMethod.
- Add optional screenshot capture on failure.
- Keep code modular and reusable.

Test Scenarios to Automate

ID	Scenario Description	Type
TC01	Verify homepage loads and dropdowns visible	Smoke
TC02	Search flights with valid cities	Functional
TC03	Complete a flight booking	Functional
TC04	Multiple bookings with different data sets	Data-driven
TC05	Blank credit card	Negative
TC06	Invalid credit card characters	Negative
TC07	Same departure and destination city	Negative

Suggested Folder Structure

```
blazedemo-automation/
├── src/
│   └── test/java/
│       ├── pages/          --> Page Object classes
│       ├── tests/          --> TestNG test classes
│       └── utils/          --> Utility classes (optional)
├── pom.xml                 --> Maven build file
├── testng.xml              --> Test suite file
└── README.md               --> Project guide
```

Expected Learning Outcomes:

- Design scalable Selenium frameworks using Page Object Model.
- Apply TestNG annotations, assertions, and data providers.
- Use Maven for dependency management.
- Write and organize positive & negative test cases.
- Gain practical skills to prepare for real-world automation tasks.